

Praktikum Design Pattern

AHMAD AFIF

SOLID: Dependency Inversion Principle

D: Dependency Inversion

Pada Prinsip dependency inversion, Class yang kita buat tidak boleh bergantung pada concrete class atau function melainkan harus bergantung pada Abstract Class dan Interface.

Prinsip ini berkesinambungan dengan prinsip kedua atau Open/Closed. Dalam Open/Closed, kode dasar tidak boleh diubah melainkan di extend atau diperluas, sedangkan disini abstract class/interface merupakan entitas tertinggi dimana kita perlu meng-extend atau meng-implement dari class tersebut untuk membuat subclass. Jadi, dapat disimpulkan bahwa kita tidak bergantung pada class yang lebih rendah, dan kita tidak mengubah kode dasar untuk mengembangkan fungsionalitas tersebut.

Entities must depend on abstractions, not on concretions. It states that the high-level module must not depend on the low-level module, but they should depend on abstractions.

D: Dependency Inversion

Seperti yang dijelaskan sebelumnya, Class yang memiliki tingkatan lebih tinggi tidak boleh bergantung pada class atau function di tingkatan yang lebih rendah maupun tingkatan yang sama. Detail dari Class tersebut harus bergantung pada abstraction maupun interface.

Gambar disamping menunjukkan violation pada konsep DIP, kita anggap lampu merupakan Class yang memiliki tingkatan lebih tinggi dibandingkan sumber energi. Ketika lampu disolder ke dinding, maka lampu akan sangat bergantung pada colokan di dinding. Jika colokan rusak, maka lampu akan kehilangan sumber energi satu-satunya.

Sekarang, coba bandingkan dengan lampu yang tidak disolder. Lampu tidak bergantung pada 1 sumber energi kongkrit tapi sumber energi abstract dimana lokasinya bisa ada dimana saja.

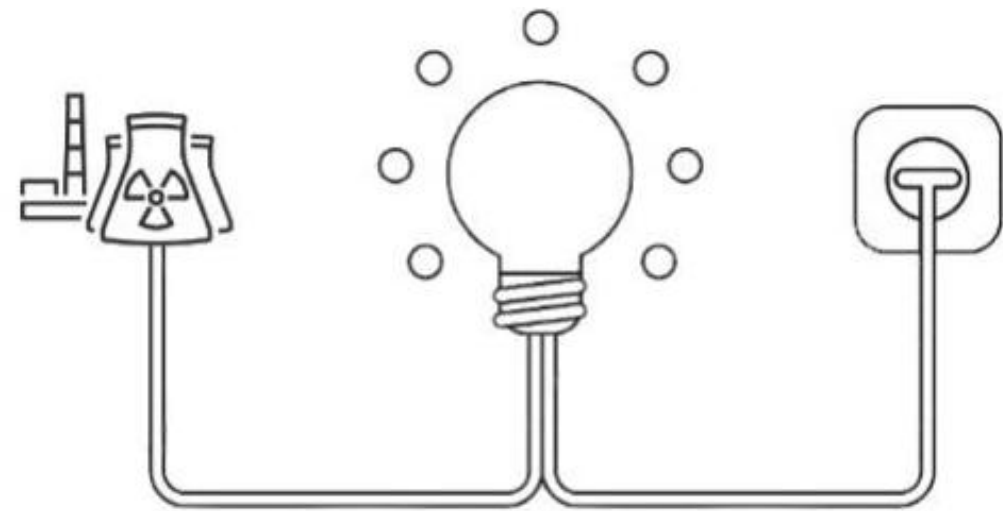


Dependency Inversion Principle

Would you solder a lamp directly to the electrical wiring in a wall?

D: Dependency Inversion

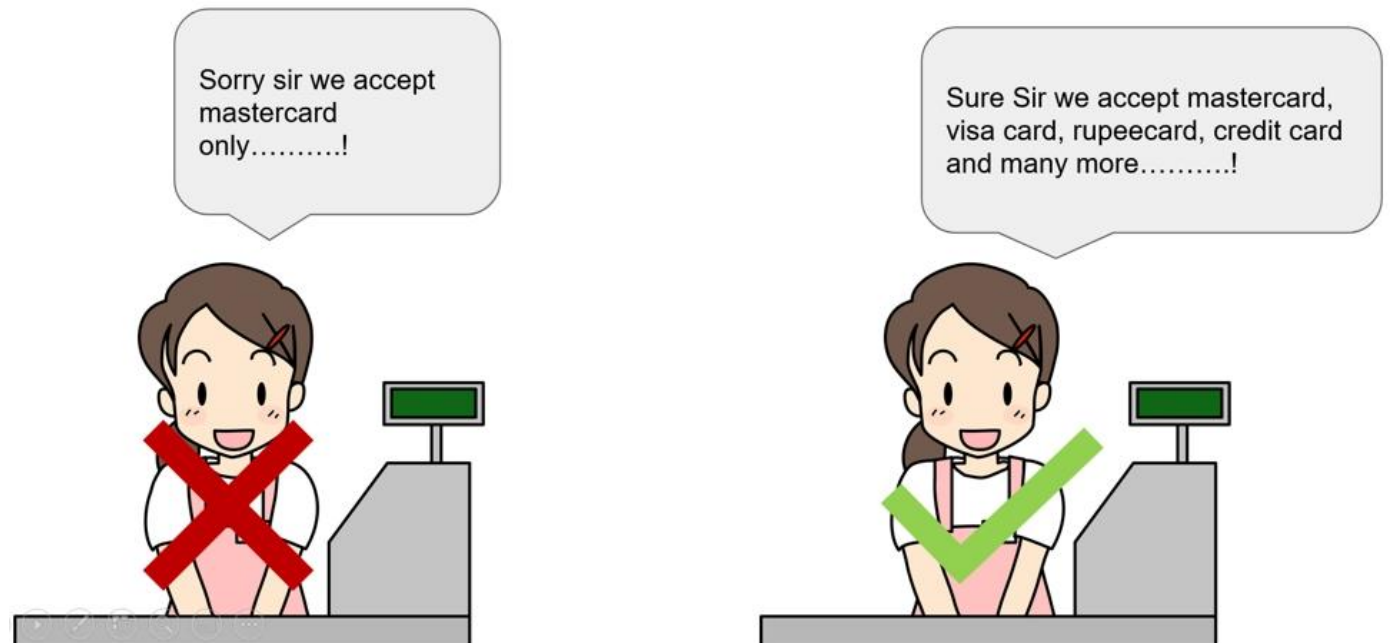
Berikut adalah contoh lain dari DIP. Lampu yang bisa menyala dengan sumber energi dari manapun, dan pembayaran yang bisa menggunakan jenis apapun (tidak terpaku pada 1 jenis saja)



Dependency Inversion Principle

When knowing how things work becomes a burden

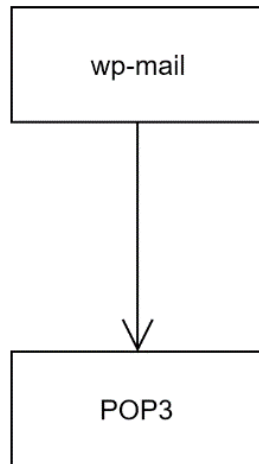
Dependency Inversion Principle



D: Dependency Inversion

Setelah melihat contoh sebelumnya, kira-kira penerapan dari DIP adalah sebagai berikut. "Class lebih tinggi tidak boleh bergantung pada class yang lebih rendah namun harus bergantung pada interface, begitupun Class yang lebih rendah harus bergantung pada interface".

Without Dependency Inversion



With Dependency Inversion

